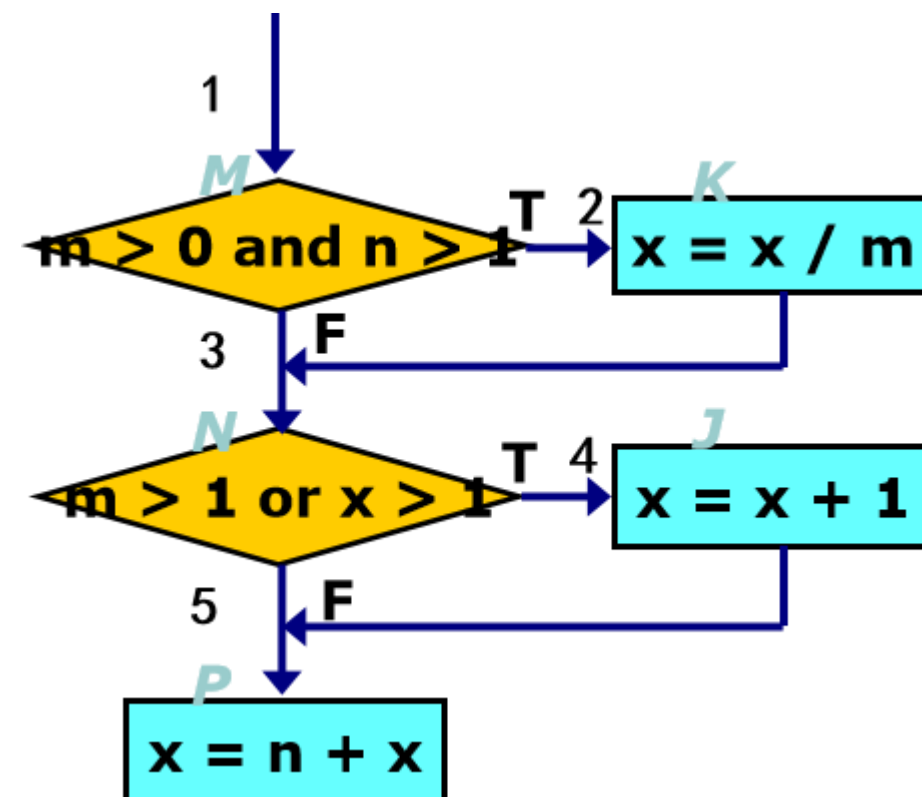


一、应用题（共 2 题，每小题 20.0 分，共 40.0 分）

1、某烟酒商店主要销售某种品牌的白酒、红酒和啤酒。白酒、红酒和啤酒的单价分别为 500 元、600 元和 20 元，每瓶白酒的利润为 150（50 瓶以上），200（50 瓶以内）元；每瓶红酒的利润为 200（100 瓶以上），300 元（100 瓶以内）；每瓶啤酒的利润为 5 元（500 瓶以上），10 元（500 瓶以内）。商店每月最多只允许销售 100 瓶白酒、200 瓶红酒和 1000 瓶啤酒。根据销售业绩，商店每月计算利润收入。使用一般边界值分析法进行测试用例的设计

2、对下列程序进行判定条件覆盖



二、测试编程题（共 3 题，每小题 20.0 分，共 60.0 分）

1、现有一个 Factorial 类用于计算阶乘结果。作为测试工程师，你需要编写参数化测试来验证该方法的正确性。被测试类代码如下（请勿修改）：

```
public class Factorial {  
    public long fact(long n) {  
        long r = 1;  
        for (long i = 1; i <= n; i++) {  
            r = r * i;  
        }  
        return r;  
    }  
}
```

要求：

- (1) 编写一个偶数参数化测试类，包含至少 2 个偶数（5 分）
- (2) 编写一个奇数参数化测试类，包含至少 2 个奇数（5 分）
- (3) 编写一个测试套件类 FactorialTestSuit，将上述两个测试类包含在内，使得运行该套件可以一次性执行所有测试。（2 分）
- (4) 提交全部 3 个测试类代码以及测试套件类运行结果截图。（3 分）

2、有如下数组处理类请不要修改 ArrayUtils 类的任何代码，只需编写测试类对其进行验证并截图测试运行结果。

```
public class ArrayUtils {

    public int findMax(int[] arr) {

        if (arr == null || arr.length == 0) {

            throw new IllegalArgumentException("数组不能为 null 或空");

        }

        int max = arr[0];

        for (int num : arr) {

            if (num > max) {

                max = num;

            }

        }

        return max;

    }

    public int findMin(int[] arr) {

        if (arr == null || arr.length == 0) {

            throw new IllegalArgumentException("数组不能为 null 或空");

        }

        int min = arr[0];

        for (int num : arr) {

            if (num < min) {

                min = num;

            }

        }

    }

}
```

```
    }

    }

    return min;

}

public int sum(int[] arr) {

    if (arr == null) {

        return 0;

    }

    int total = 0;

    for (int num : arr) {

        total += num;

    }

    return total;

}

public int average(int[] arr) {

    return sum(arr) / arr.length;

}

}
```

要求:
使用@Before 注解初始化 ArrayUtils 以及用于测试的数组 {67, 434, 98, 2, 4}。
使用@After 注解把测试对象与测试数组置为 null 释放资源。
测试最大值
测试最小值
测试求和

测试求平均

提交整个测试类代码以及测试运行结果截图。

3、有如下计算器类 `Calculator`，请不要修改 `Calculator` 类的任何代码，只需编写测试类对其进行验证并截图测试运行结果。

```
public class Calculator {  
  
    public static void main(String[] args) {  
  
    }  
  
    public int add(int a, int b) {  
  
        return a + b; // 简单的加法运算  
  
    }  
  
    public int minus(int a, int b) {  
  
        return a - b; // 减法运算  
  
    }  
  
    public int multi(int a, int b) {  
  
        return a * b; // 乘法运算  
  
    }  
  
    public int divd(int a, int b) throws Exception {  
  
        return a / b; // 除法运算  
  
    }  
  
}
```

要求:

- (1) 使用 `@Before` 注解初始化 `Calculator`。
- (2) 使用 `@After` 注解把测试对象置为 `null` 释放资源。
- (3) 测试加减乘除操作
- (4) 提交整个测试类代码以及测试运行结果截图。